

A surrogate for the carried code: RBLapSum on the branch, the carry left hard

8-layer TinyStories models of the K+J notes, `code_residual`, abs-TopK over $N = 1536$, $T = 2$, MD optimizer, one seed; runs `be_cr_*`, configs `configs/cr_kj/carry/`, `rblapsum_surrogate_scope=carry_*`.

1 The problem

With the code carried between blocks hard Top- K' gains 0.06–0.075 nats at every K' and RBLapSum (K active, pool $K+J$) gains -0.03 to $+0.015$: at a matched active count it is worth 0.047 at $K = 32$, loses 0.02–0.03 at $K = 128, 256$, no cell reaches hard Top- $(K+J)$, and its backward is the unstable part of the grid (`docs/kj-vs-hard-topk-code-residual.tex`); the first-order scope removes the instability at the price of a second backward pass and is *worse* than it at every $K = 32$ cell. This note removes the conflict at no cost.

2 Where the probabilities act

Gate ℓ reads $u_\ell = c_\ell + v_\ell$ (carried code plus the block’s encoded contribution) and outputs $c_{\ell+1} = M_\ell u_\ell$, M_ℓ the mask. Its output has two consumers, the carry $u_{\ell+1} = c_{\ell+1} + v_{\ell+1}$ (a skip path) and the read $x_{\ell+1} = N(D_\ell c_{\ell+1})$ (a branch):

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = M_\ell (g_{\ell+1}^c + r_{\ell+1}), \quad g_{\ell+1}^c = \frac{\partial \mathcal{L}}{\partial u_{\ell+1}}, \quad r_{\ell+1} = D_\ell^\top \frac{\partial \mathcal{L}}{\partial (D_\ell c_{\ell+1})} \quad (1)$$

(r is the unembedding’s at the last gate). In the stream-carried stack the gate has only the second consumer: everything block $\ell+1$ does, its own gate included, goes through D_ℓ . RBLapSum relaxes the gate as a whole, $M_\ell \rightarrow M_\ell + S_\ell$ (kernel $\kappa_i = e^{-|s_i - b|/T}/2T$; $S_{ii} = |u_i| \kappa_i \approx b/2T$ at the boundary). In the stream stack S_ℓ acts on $r_{\ell+1}$ alone and reaches gate $\ell-1$ through E_ℓ , the block and $D_{\ell-1}$, which mix the coordinates. In the code stack it also acts on $g_{\ell+1}^c$, which already contains $S_{\ell+1} g_{\ell+2}^c$: along a coordinate near the boundary the backward multiplies $1 + |u| \kappa$ per gate. The carry is a residual connection in code space, trainable because its backward is the identity; a relaxation on the skip path compounds. The conflict is *where* the probabilities act, not the kernel.

3 The carry-aware surrogate

Keep the skip path hard, relax the branch: the gate’s output is split into the copy the next gate carries and the copy the decoder reads (forward and inference unchanged), and

$$\frac{\partial \mathcal{L}}{\partial u_\ell} = M_\ell (g_{\ell+1}^c + r_{\ell+1}) + S_\ell^\top \hat{g}_{\ell+1}, \quad \hat{g}_{\ell+1,i} = r_{\ell+1,i} + \begin{cases} 0 & \text{local} \\ H_{\ell+1,i} & \text{hard} \\ \rho R_{\ell+1,i} & \text{persistent,} \end{cases} \quad (2)$$

with $H_\ell = M_\ell (r_{\ell+1} + H_{\ell+1})$ the downstream read gradients a coordinate reaches along the hard carry, $R_\ell = r_{\ell+1} + \rho R_{\ell+1}$ every downstream read with a survival factor ρ per gate, $H = R = 0$ above the last gate. The support term is driven by read gradients only, so no kernel sees another gate’s support term through the carry: the products are gone, the terms of the gates above still reach gate ℓ added through M_ℓ , carry and update receive one gradient (R1 of `docs/rblapsum-code-residual.tex`); the cost is two per-coordinate sums along the backward. The drivers are the first-order loss change of one flip at gate ℓ ($c_{\ell+1,i}$ changes by $\pm u_{\ell,i}$; block $\ell+1$ reads it, later blocks while the coordinate stays in the code) under three models of what follows. *Local*: the next read only – the stream stack’s logic, exactly the gradient of a model whose decoder reads a relaxed gate while the carry is hard. *Hard*: the actual downstream masks (nothing, for most inactive candidates). *Persistent*: an entering coordinate survives each further gate with probability ρ . *Mixed*: hard for the active members, persistent for the inactive ones.

4 Measurements

Setup. The code-carried $T = 2$ arm of the K+J grid (no output norm, its 20k-step schedule and seed). Baselines are the grid’s runs read at matching steps; each new scope trains the grid’s config with one field changed, stopped at step 3000 with the schedules untouched, two cells continued to 10k. At step 3000 the grid’s surrogate margins are largest and already of their final sign (pool, 3k→20k: $-0.061 \rightarrow -0.042$ at $K = 32$, $+0.048 \rightarrow +0.034$ at $K = 256$).

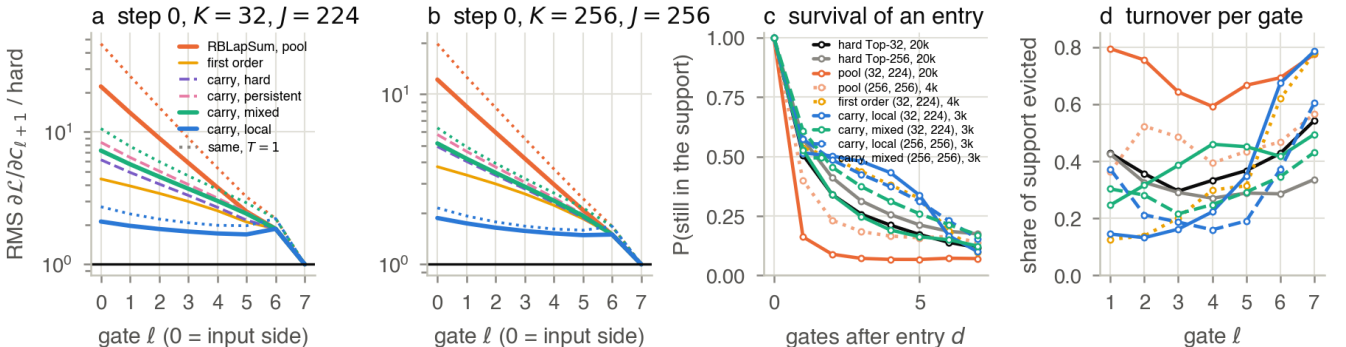


Figure 1: a, b Backward along the carried code at initialization: RMS $\partial \mathcal{L} / \partial c_{\ell+1}$ at each gate’s output relative to the hard backward. Under pool it grows $\times 1.5$ ($K=32$) and $\times 1.4$ ($K=256$) per gate toward the input, $22\times$ and $12\times$ hard at gate 0 ($46\times$ at $T=1$, dotted); the carry scopes and first order grow additively. c, d Runs of the code: $P(\text{an entered feature is still in the support } d \text{ gates later})$; share of the support gate ℓ receives that it evicts.

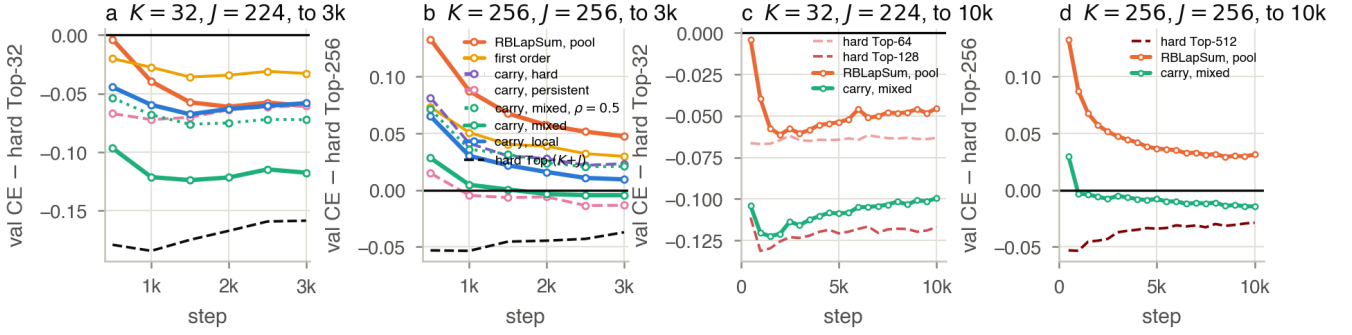


Figure 2: Validation CE relative to hard Top-K: a, b the first 3000 steps, dashed hard Top-(K+J); c, d the mixed scope continued to 10k against the grid’s 20k runs.

At initialization (Fig. 1a,b) the surrogate’s share of the gradient energy at the gates, $\sigma = \|S^\top \hat{g}\|^2 / \|m \odot g\|^2$, is 0.70 for pool, 0.74 for mixed, 0.31 for local and 0.24 for first order at $K = 32$: mixed injects as much support signal as the unmodified backward, as a sum.

Runs of the carried code (Fig. 1c,d; 16 validation sequences, hard forward). The carry persists for a subset of the code: in the trained hard Top-32 model every gate evicts 30–54% of the support it receives, an entered feature is still there one gate later with probability 0.51 and two later 0.34, 12% of the final code was carried from gate 0 and 54% entered at the last gate (hard Top-256: 0.57, 0.41, 17%, 34%); an entry at gate 0 is read by 2.7–3.1 blocks on average (persistent model: 8, local: 1). The pool surrogate trains the model *out of* the carry: at (32, 224) turnover 63% per gate at step 4000 and 70% at 20k, survival $0.21 \rightarrow 0.16$, 77% of the final code entering at the last gate – the stream stack’s re-selection at every block. Every carry-scope model keeps the hard model’s code (survival 0.52–0.61, turnover 30–40% at 3000; the mixed (32, 224) and (256, 256) models at 10k: 0.47/45%, 0.59/33%; hard at 20k: 0.51/39%, 0.57/32%).

Training (Fig. 2). At (32, 224) the local scope ends step 3000 at -0.058 below hard Top-32, where pool is at -0.061 and first order at -0.033 . The mixed scope ends at -0.118 , twice the margin of the unmodified surrogate and the margin of hard Top-128 over hard Top-32 at that step (-0.124): 32 active features at the loss of the hard model that activates 128 (1.782 against 1.776). Continued to 10k its margin decays as the grid’s do: -0.100 at 10k, against hard Top-128 -0.117 , hard Top-64 -0.063 , pool -0.045 . At (256, 256), the pathological cell (pool $+0.047$ at 3000, turnover 46% per gate against the hard model’s 28%), the local scope closes the gap to $+0.009$ and the mixed scope is below hard Top-256 from step 2000, -0.0045 at 3000 (1.736 against 1.741): a surrogate ahead of the hard model with the same active count (hard Top-512 -0.037 ; the hard baselines also have the post-norm, worth about 0.007 in the stream grid). Continued to 10k the lead widens to -0.014 (pool $+0.031$, hard Top-512 -0.029). On the diagonal $J = K$ the mixed scope is below hard Top- K from step 500 on and ends at -0.047 at (64, 64) (pool -0.005 , first order -0.003 ; hard Top-128 -0.059) and -0.006 at (128, 128) (pool $+0.025$, first order $+0.026$; hard Top-256 -0.035). At (128, 384) mixed ends at -0.025 (pool $+0.016$, first order $+0.021$). The drivers: at (256, 256) persistent on both sides -0.013 , mixed -0.005 , local $+0.009$, hard on both sides $+0.023$, mixed with $\rho = 0.5$ $+0.021$; at (128, 128) persistent -0.022 , mixed -0.006 ; at (32, 224) mixed -0.118 , mixed with $\rho = 0.5$ -0.072 , persistent -0.060 , local -0.058 . The entry term works best as if the candidate were carried to the end ($\rho = 1$) at every K ; the eviction term wants the actual run at $K = 32$, where the persistent driver loses the whole doubling, and the persistent sum at $K \geq 128$ (by 0.016 and 0.008) – the K -dependence of the active-side term seen in the 24-layer study. No carry-scope run’s gradient norm exceeded 1 (pool at (32, 224): four spikes, to 3.3), and their feature-usage entropy is the hard model’s (pool’s is lower).

5 What the measurements show

The conflict is the placement, not the kernel. RBLapSum relaxes a gate whose output, in the code-carried stack, is also a skip connection, and the relaxation is multiplied along it, $1 + b/2T$ per gate. Putting the probabilities where they were in the stream-carried stack – on what the decoder reads – and leaving the carry’s backward the mask removes the product exactly, in one pass; the local scope keeps the unmodified surrogate’s whole gain at $K = 32$ (first order keeps half) and removes four fifths of its loss at $K = 256$.

The carry gives the surrogate what the stream stack could not: a downstream for the entry term. In the code stack an entering feature is carried, so the first-order value of admitting it is the sum of the reads it would reach – a sum the stream stack, which re-encodes at every block, does not have. The mixed scope evaluates inactive candidates on that sum and active members on the reads their run reaches: it doubles the gain at $K = 32$ and turns the loss at $K = 256$ into a gain that is still widening at 10k, with the unmodified backward’s surrogate energy ($\sigma = 0.74$ against 0.70) delivered as a sum of single-flip terms instead of a product.

The model uses the carry only when the surrogate leaves it alone. pool-trained models re-select 46–70% of their support at every gate and end with the stream stack’s code, which is why the carry was worth nothing to them; every carry-scope model has the hard model’s survival and turnover, so the support signal and the carry’s persistence add instead of competing. **Scope:** one seed, one family (8 layers, $N = 1536$, $T = 2$, no output norm), 3000-step screens and two 10k continuations against step-matched 20k baselines; $T = 1$, the post-norm arm and deeper stacks untested.